

Project 702: PR-Driven Self-Service GitOps for OpenChoreo Developers

Architecture Overview: OpenChoreo is a modular, multi-plane developer platform for Kubernetes that provides higher-level abstractions alongside a Backstage-powered portal, CI/CD, GitOps, and observability

【15†L47-L51】 【20†L270-L278】 . In practice, the **control plane** hosts the APIs, the Backstage UI/CLI (“experience plane”), and the GitOps engine. Developers use the Backstage portal (or the `occ` CLI) to declare intent – for example, creating a new **Workload** or **ComponentRelease** CR. These high-level abstractions are Kubernetes-native (e.g. OpenChoreo CRDs) and are reconciled by controllers in the control plane into real cluster resources 【16†L62-L70】 【16†L142-L150】 . The **workflow plane** provides build and CI automation (by default using cloud-native Buildpacks and Argo Workflows 【20†L288-L291】), while the **data plane** is where user workloads actually run under enforced policies and network configurations 【16†L78-L87】 . In Project 702, the goal is to stitch these pieces together so that *Pull Requests drive the GitOps flow*: a Backstage Scaffolder action will generate OpenChoreo manifests (via `occ`), commit them to a Git repo, and create a PR. Once merged, a GitOps operator (FluxCD or Argo CD) applies those manifests to the cluster, and the control plane reconciles them into Kubernetes resources. In this way developers gain a self-service workflow without hand-crafting low-level YAML 【15†L47-L51】 【16†L142-L150】 .

【51†embed_image】 *Argo Workflows*: OpenChoreo’s workflow plane can leverage Argo Workflows to execute CI pipelines 【20†L288-L291】 . Argo Workflows is a Kubernetes-native engine supporting DAG and step-based workflows 【50†L26-L34】 , making it well-suited for building/testing components and producing container images or manifest artifacts as part of the GitOps pipeline. For example, after a developer pushes code, an Argo Workflow could build the image and then invoke `occ apply -f workload-cr.yaml` to register the new component in OpenChoreo (as shown in Jenkins example usage 【12†L5722-L5728】). This offloads the build logic from the developer back into repeatable workflows.

【52†embed_image】 *Argo CD (GitOps Operator)*: Once the PR with new OpenChoreo YAML is merged, a GitOps operator like Argo CD takes over. Argo CD is “a declarative, GitOps continuous delivery tool for Kubernetes” 【32†L285-L293】 that continuously monitors the Git repository and applies the specified manifests to target clusters. In practice, Argo CD (or FluxCD) will detect the merged OpenChoreo CRs in the repo and apply them to the cluster. The OpenChoreo control plane then sees the new Workload/ComponentRelease definitions and reconciles them into actual Kubernetes Deployments/Services, completing the flow from PR to running workload. This end-to-end GitOps loop ensures that developers’ PRs (not the pipeline jobs themselves) drive the system state, giving visibility and auditability through Git and the portal.

6-Month Implementation Roadmap

To deliver Project 702 within six months, the work can be organized into phased milestones. The following steps build on an existing local setup (OpenChoreo on k3d with Argo CI):

- 1. Foundation & Exploration (Weeks 1–4):** Deploy OpenChoreo locally (e.g. Quick Start using Docker+K3d [14†L162-L166]) and verify the Backstage portal is accessible. Experiment with `occ` commands to manually create or apply a Workload and ComponentRelease CR [12†L5722-L5728] . Study the existing workflow (Argo Workflows) that builds and registers components, noting how `occ apply` simplifies pipelines (no manual `kubectl` calls) [12†L5722-L5728] . Document the current GitOps workflow (if any) in the repository.
- 2. Scaffolder Prototype (Weeks 5–8):** Set up a Backstage Scaffold integration. Create a minimal template and custom action that takes user input (e.g. service name, image) and runs `occ workload create/apply` to generate a YAML manifest. This could write a `Workload` or `ReleaseBinding` CR file. Verify locally that the action produces a valid YAML (e.g. by inspecting with `kubectl get workload`). Ensure the action can run in the portal's backend (may require copying OCC client configuration or secrets). At this stage, just output the manifest to a workspace directory – focus on getting `occ` invocation right.
- 3. Git Repo Layout & Commit (Weeks 9–12):** Design the GitOps repository structure (e.g. folders by environment or component). Implement a resolver in the scaffold that determines the correct path to commit the new YAML. Extend the scaffold action to initialize a Git branch (or repository) with the manifest files. Use an existing Backstage GitHub/GitLab action or write a custom script to clone the repo, copy in the generated YAML, and push to a new branch. Ensure the branch naming and file paths can be configured (e.g. one branch per PR, or one environment per branch).
- 4. Automated PR Creation (Weeks 13–16):** Integrate with the Git provider's API to automatically open a Pull Request. For example, use GitHub/GitLab tokens to commit and create a PR from the scaffold action. The PR should include the generated YAML and a descriptive title (e.g. "feat(BackendService): register new workload"). Configure the portal to display PR status (Backstage scaffold can emit hyperlinks or use the Git provider plugin). At this point, test the loop: run scaffold → generates files → push → open PR.
- 5. GitOps Sync & Reconciliation (Weeks 17–20):** Deploy a GitOps operator in the cluster. For example, install Argo CD and connect it to the GitOps repository (monitoring the main branch or a specific path). Merge a test PR, and verify that Argo CD detects it and syncs to the cluster. Check that the new OpenChoreo CRs appear in `kubectl get` and that the corresponding Kubernetes resources (Deployment, Service, etc.) are created by the control plane. Add back-port logic so that the portal can query OpenChoreo for component status (to show "synced" vs "out of sync"). Ensure authentication: store the Git credentials (token for pushing, and token or SSH key for Argo CD to pull) securely (e.g. Kubernetes secrets).

This step completes the core flow: PR → merge → Argo CD sync → workload active. By merging via Git, all changes remain auditable. (Note that Argo CD continuously enforces Git as the source of truth [32†L316-L324] , so manual edits to the cluster will be reverted, keeping the desired state consistent.)

1. **Advanced Features & Testing (Weeks 21–24):** Improve and harden the solution. Add **conflict detection**: for example, if two PRs modify the same Workload resource, the portal or scaffolder should warn the user. Implement **configurable repo layout** (monorepo vs multiple repos) so that enterprise teams can adapt it. Extend the scaffolder to support other OpenChoreo CRs (like `ComponentRelease` or `ReleaseBinding`), and add hybrid dependency resolution (resolve `ComponentType` and `Trait` references from both platform config and the workspace). Throughout, follow an incremental approach: build one feature at a time and reuse common plumbing (as one developer noted, “The migration is incremental by design: one component, one extra stage, one pipeline you don’t otherwise touch” [12†L5791-L5799]).

Finally, write **end-to-end tests** (for example, a CI job that uses the REST API to create a portal user, runs the scaffolder, and checks that the new app is deployed in Kubernetes). Prepare documentation and a demo script: show a developer changing code, using the portal to create a PR, and observing the new service appear in `kubect1` and the OpenChoreo portal. With these milestones, by month 6 you will have a fully automated PR-driven GitOps workflow in OpenChoreo, along with documentation and examples.

Sources: OpenChoreo documentation and examples [15†L47-L51] [16†L62-L70] [20†L288-L291] ; Backstage/Scaffolder guides (conceptual); Argo Project documentation [50†L26-L34] [32†L285-L293] ; and related design discussions [12†L5722-L5728] [12†L5791-L5799] .
